

```

#!/usr/bin/env python3
"""
Jovem (Y1, Y2) vs. Idoso (O1) – HRV real
(Fantasia/PhysioNet) + 30 sementes
"""

import numpy as np
from scipy.interpolate import interp1d
from scipy import stats

K = 4
T = 400
kappa = 0.25
precision_ema = 0.05
ignition_threshold = 2.5
broadcast_gain_normal = 0.5
obs_noise = 0.3
precision_ceiling_normal = 5.0
N_SEEDS = 30

def true_signal(k, t):
    base_freqs = [0.02, 0.035, 0.015, 0.05]
    event_times = [100, 180, 260, 320]
    val = np.sin(2 * np.pi * base_freqs[k]
    * t)
    if t > event_times[k]:
        val += 1.5
    return val

def sliding_rmssd(rr_series, window=60,
step=30):

```

```

        diffs = np.diff(rr_series) * 1000
        vals = []
        for i in range(0, len(diffs) - window,
step):
            w = diffs[i:i+window]
            vals.append(np.sqrt(np.mean(w**2)))
        return np.array(vals)

def hrv_to_refractory(hrv_series,
min_ref=8, max_ref=30):
    hrv_norm = (hrv_series -
hrv_series.min()) / (hrv_series.max() -
hrv_series.min() + 1e-9)
    return np.round(max_ref - (max_ref -
min_ref) * hrv_norm).astype(int)

def load_subject(path):
    rr = np.loadtxt(path)
    rmssd = sliding_rmssd(rr)
    x_old = np.linspace(0, 1, len(rmssd))
    x_new = np.linspace(0, 1, T)
    rmssd_interp = interp1d(x_old, rmssd,
kind='linear')(x_new)
    return rr, rmssd, rmssd_interp

base =
'/home/claude/active_inference_workspace/'
rr_y1, rmssd_y1, hrv_y1 = load_subject(base
+ 'fantasia_Y1_RR_intervals.txt')
rr_y2, rmssd_y2, hrv_y2 = load_subject(base
+ 'fantasia_Y2_RR_intervals.txt')
rr_o1, rmssd_o1, hrv_o1 = load_subject(base
+ 'fantasia_O1_RR_intervals.txt')

```

```

print("="*70)
print("HRV REAL: JOVEM (Y1,Y2) vs. IDOSO
(01) – Fantasia/PhysioNet")
print("="*70)
print(f"Y1: RMSSD medio =
{rmssd_y1.mean():.1f} ms (n={len(rmssd_y1)}
janelas)")
print(f"Y2: RMSSD medio =
{rmssd_y2.mean():.1f} ms (n={len(rmssd_y2)}
janelas)")
print(f"01: RMSSD medio =
{rmssd_o1.mean():.1f} ms (n={len(rmssd_o1)}
janelas)")
print(f"\nMedia jovem (Y1+Y2):
{np.mean([rmssd_y1.mean(),
rmssd_y2.mean()]):.1f} ms")
print(f"Media idoso (01):
{rmssd_o1.mean():.1f} ms")

refractory_y1 = hrv_to_refractory(hrv_y1)
refractory_y2 = hrv_to_refractory(hrv_y2)
refractory_o1 = hrv_to_refractory(hrv_o1)

def run_simulation(seed,
refractory_series=None,
refractory_fixed=15):
    np.random.seed(seed)
    mu = np.zeros(K)
    precision = np.ones(K) * 1.0
    err_ema_sq = np.ones(K) * 1.0
    refractory_until = np.zeros(K,
dtype=int)
    ignitions = 0
    abs_err_sum = 0.0

```

```

        for t in range(T):
            ref_period = refractory_series[t]
            if refractory_series is not None else
            refractory_fixed
                true_vals =
np.array([true_signal(k, t) for k in
range(K)])
            obs = true_vals +
np.random.randn(K) * obs_noise
            err = obs - mu
            salience = precision * err**2
            mu = mu + kappa * precision * err
            err_ema_sq = (1 - precision_ema) *
err_ema_sq + precision_ema * err**2
            precision = np.clip(1.0 / (0.1 +
err_ema_sq), 0.1, precision_ceiling_normal)
            eligible = np.array([salience[k] if
t >= refractory_until[k] else -1.0 for k in
range(K)])
            winner = int(np.argmax(eligible))
            ignite = eligible[winner] >
ignition_threshold
            if ignite:
                ignitions += 1
                refractory_until[winner] = t +
ref_period
                for j in range(K):
                    if j != winner:
                        mu[j] = mu[j] +
broadcast_gain_normal * (mu[winner] -
mu[j])
                abs_err_sum += np.mean(np.abs(mu -
true_vals))

```

```

        return ignitions, abs_err_sum / T

def run_multi_seed(refractory_series=None,
refractory_fixed=15, n_seeds=N_SEEDS):
    ign_list, err_list = [], []
    for s in range(n_seeds):
        ign, err =
run_simulation(seed=1000+s,
refractory_series=refractory_series,
refractory_fixed=refractory_fixed)
        ign_list.append(ign)
        err_list.append(err)
    return np.array(ign_list),
np.array(err_list)

print("\n" + "="*70)
print(f"RODANDO {N_SEEDS} SEMENTES POR
CONDICAO")
print("="*70)

ign_fixed, err_fixed =
run_multi_seed(refractory_series=None)
ign_y1, err_y1 =
run_multi_seed(refractory_series=refractory
_y1)
ign_y2, err_y2 =
run_multi_seed(refractory_series=refractory
_y2)
ign_o1, err_o1 =
run_multi_seed(refractory_series=refractory
_o1)

# agrupar jovem = media dos dois sujeitos

```

```

jovens, por semente
err_young = (err_y1 + err_y2) / 2
ign_young = (ign_y1 + ign_y2) / 2

def report(name, ign, err):
    print(f"\n{name}:")
    print(f" Ignicoes: media=
{ign.mean():.1f} desvio-padrao=
{ign.std():.1f}")
    print(f" Erro: media=
{err.mean():.4f} desvio-padrao=
{err.std():.4f}")

report("REFRATARIO FIXO (15 passos)",
ign_fixed, err_fixed)
report("REFRATARIO via HRV JOVEM (media
Y1+Y2)", ign_young, err_young)
report("REFRATARIO via HRV IDOSO (01)",
ign_o1, err_o1)

print("\n" + "="*70)
print("TESTES ESTATISTICOS (jovem vs.
idoso, pareado por semente)")
print("="*70)

t_err, p_err = stats.ttest_rel(err_young,
err_o1)
t_ign, p_ign = stats.ttest_rel(ign_young,
ign_o1)
print(f"Erro medio: jovem=
{err_young.mean():.4f} vs idoso=
{err_o1.mean():.4f} | t={t_err:.3f}, p=
{p_err:.4f}")
print(f"Ignicoes: jovem=

```

```
{ign_young.mean():.1f} vs idoso=
{ign_o1.mean():.1f} | t={t_ign:.3f}, p=
{p_ign:.4f}")

t_err2, p_err2 = stats.ttest_rel(err_fixed,
err_young)
print(f"\nFixo vs. jovem (erro): t=
{t_err2:.3f}, p={p_err2:.4f}")
t_err3, p_err3 = stats.ttest_rel(err_fixed,
err_o1)
print(f"Fixo vs. idoso (erro): t=
{t_err3:.3f}, p={p_err3:.4f}")
```